

Doc. no. D0174R0
Date: 2016-02-15
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Deprecating Vestigial Library Parts in C++17

Table of Contents

- [Revision History](#)
 - [Revision 0](#)
- [Introduction](#)
- [Recommendations for Deprecation](#)
 - [Deprecate `std::iterator`](#)
 - [Deprecate `value\_compare` Predicates](#)
 - [Deprecate algorithms taking half an input range](#)
 - [Deprecate the Redundant Members of `std::allocator`](#)
- [Additional Candidates](#)
 - [Reconsider `vector<bool>` Partial Specialization](#)
 - [Reconsider `is\_literal` Trait](#)
 - [Reconsider the Temporary Buffer APIs](#)
 - [Reconsider raw storage iterators](#)
- [Proposed Wording](#)
 - [20.7.2 Header `<memory>` synopsis \[memory.syn\]](#)
 - [20.7.9 The default allocator \[default.allocator\]](#)
 - [20.7.9.1 allocator members \[allocator.members\]](#)
 - [23.2.4 Associative containers \[associative.reqmts\]](#)
 - [23.4.4 Class template `map` \[map\]](#)
 - [23.4.4.1 Class template `map` overview \[map.overview\]](#)
 - [23.4.5 Class template `multimap` \[multimap\]](#)
 - [23.4.5.1 Class template `multimap` overview \[multimap.overview\]](#)
 - [23.4.6 Class template `set` \[set\]](#)
 - [23.4.6.1 Class template `set` overview \[set.overview\]](#)
 - [23.4.7 Class template `multiset` \[multiset\]](#)
 - [23.4.7.1 Class template `multiset` overview \[multiset.overview\]](#)
 - [24.3 Header `<iterator>` synopsis \[iterator.synopsis\]](#)
 - [24.4.2 Basic iterator \[iterator.basic\]](#)
 - [25 Algorithms library \[algorithms\]](#)
 - [25.1 General \[algorithms.general\]](#)
 - [25.2.10 Mismatch \[mismatch\]](#)
 - [25.2.10 Mismatch \[mismatch\]](#)
 - [25.2.11 Equal \[alg.equal\]](#)
 - [25.2.12 Is permutation \[alg.is_permutation\]](#)
 - [D.w The default allocator \[depr.default.allocator\]](#)
 - [D.x Value Comparators for Associative Containers \[depr.container.assoc.value_compare\]](#)
 - [D.y Basic iterator \[depr.iterator.basic\]](#)
 - [D.z Algorithms with Partially Formed Ranges \[depr.algorithm.partial.range\]](#)
- [Acknowledgements](#)
- [References](#)

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Introduction

A number of features of the C++ Standard library have been surpassed by additions over the years, or we have learned do not serve their intended purpose as well as originally expected. This paper proposed deprecating features where better, simpler, or clearer options are available.

Recommendations for Deprecation

There is likely to be a reasonable amount of user code using all of the feature below in production today, so it would be premature to remove them from the standard. However, with best best practice advancing and superior or clearer options available within the standard itself, now would be a good time to deprecate these features, directing users to the preferred alternative instead.

Deprecate `std::iterator`

As an aid to writing iterator classes, the original standard library supplied the `iterator` class template to automate the declaration of the five typedefs expected of every iterator by `iterator_traits`. This was then used in the library itself, for instance in the specification of `std::ostream_iterator`:

```
template <class T, class charT = char, class traits = char_traits<charT> >
class ostream_iterator:
    public iterator<output_iterator_tag, void, void, void, void>;
```

The long sequence of `void` arguments is much less clear to the reader than simply providing the expected typedefs in the class definition itself, which is the approach taken by the current working draft, following the pattern set in C++14 where we deprecated the derivation throughout the library of functors from `unary_function` and `binary_function`.

In addition to the reduced clarity, the `iterator` template also lays a trap for the unwary, as in typical usage it will be a dependent base class, which means it will not be looking into during name lookup from within the class or its member functions. This leads to surprised users trying to understand why the following simple usage does not work:

```
#include <iterator>

template <typename T>
struct MyIterator : std::iterator<std::random_access_iterator_tag, T> {
    value_type data; // Error: value_type is not found by name lookup

    // ... implementations details elided ...
};
```

The reason of clarity alone was sufficient to persuade the LWG to update the standard library specification to no longer mandate the standard iterator adaptors as deriving from `std::iterator`, so there is no further use of this template within the standard itself. Therefore, it looks like a strong candidate for deprecation.

Deprecate `value_compare` predicates

The standard containers `map` and `multimap` both contain an identical class template `value_compare` that is a functor which can be used to determine the relative order of two elements in that container. These are provided as elements in a map are ordered by only the key value, rather than the whole value, so the predicate supplied to the container compares the key type, not the actual element type.

In practice these functors are not used by the containers, as they are often required to compare keys with values for elements that have not yet been inserted into the map, and may not have been created as a `pair` yet. Meanwhile, the cost of having a nested class type, rather than an alias to a functor with the same behavior, means that every associative container with a different allocator has an identical copy of this class, but with a different name mangling so that the duplicate functionality cannot be elided or merged. The class name itself will have a longer mangling than necessary, which would add up if these functors were genuinely useful and saw much use in practice.

However, it is not clear that these functors do see much actual use. For example, a quick search with Google Code search while preparing this paper turned up 29 hits, which were all standard library implementations, or test drivers for standard library implementations, and a similar result for the `value_comp()` function that returns an appropriately constructed functor from the container for the client to use.

This paper recommends deprecating the `value_compare` member classes of the associative containers, and moving their declarations and definitions to Annex D, just as the old `iostreams` members were handled in the original 1998 standard. It further recommends making it unspecified whether these members are provided as member-classes, per the text of the standard, or as aliases to a class with the same interface, allowing, but not requiring, vendors to choose to provide these members in a less redundant (but ABI-breaking) manner.

Deprecate algorithms taking half an input range

The standard library has several algorithms that read from two ranges in order to determine their result. In most cases, the original C++98 standard fully specified the first range with a pair of iterators, and supplied only the first iterator for the second range, with a narrow contract requirement that the second range be at least as large as the first. For the C++14 standard, these algorithms were overloaded with a second form where the second range is also fully specified by a pair of iterators, partly so that the length of the second sequence can be taken into account (when it is shorter), and partly because this form is less liable to misuse leading to buffer overruns and other security risks. See [N3671](#) **Making non-modifying sequence operations more robust** for more details.

Given the security risks associated with the original algorithms, this paper recommends deprecating the form of any algorithm where a second input range is specified by only one iterator. Note that algorithms using a single iterator for an output range continue to be supported, as there is no (standard) way to specify the end of an output iteration sequence, such as defined by an `std::ostream_iterator`.

Deprecate the redundant members of `std::allocator`

Many members of `std::allocator` redundantly duplicate behavior that is otherwise produced by `std::allocator_traits<allocator<T>>`, and could safely be removed to simplify this class. Further, `addressof` as a free function supersedes `std::allocator<T>::address` which requires an allocator object of the right type. Finally, the reference type aliases were initially provided as an expected means for extension with other allocators, but turned out to not serve a useful purpose when we specified the allocator requirements (17.6.3.5 [allocator.requirements]).

While we cannot remove these members without breaking backwards compatibility with code that explicitly used this allocator type, we should not be recommending their continued use. If a type wants to support generic allocators, it should access the allocator's functionality through `allocator_traits` rather than directly accessing the allocator's members - otherwise it will not properly support allocators that rely on the traits to synthesize the default behaviors. Similarly, if a user does not intend to support generic allocators, then it is much simpler to directly invoke `new`, `delete`, and assume the other properties of `std::allocator` such as pointer-types directly.

While the `is_always_equal` trait might be implicitly synthesized from the trait `is_empty_v<allocator<T>>`, there appears to be no specified guarantee that `std::allocator` specializations be empty classes (although this is widely expected) so the trait remains explicitly specialized to provide the necessary guarantee, even for implementations that choose to add some data to `std::allocator` for debug, profiling, or other purposes.

Similarly, `std::allocator<void>` is defined so that various template rebinding tricks could work in the original C++98 library, but it is not an actual allocator, as it lacks both `allocate` and `deallocate` member functions, which cannot be synthesized by default from `allocator_traits`. That need went away with C++11 and the `void_pointer` and `const_void_pointer` type aliases in `allocator_traits`. However, we continue to specify it in order to avoid breaking old code that has not yet been upgraded to support generic allocators, per C++11.

This paper recommends deprecating `std::allocator<void>` and the redundant members of the `std::allocator` class template, and moving their declarations and definitions to Annex D, just as the old `iostreams` members were handled in the original 1998 standard.

One potential concern is whether removing these members in the future might cause a compile-time performance regression, if the library is expected to deduce the default behavior in each case. The proposed Zombie Names clause from [P0090R0](#) and incorporated in [P0005R3](#) would allow vendors to retain these names indefinitely, if that were a concern. However, the optimal (compile-time) solution for the library implementation is to provide a partial specialization for `std::allocator_traits<std::allocator<T>>`, and so entirely avoid the cost of evaluating complex template machinery for the default allocator in the standard library.

It was also observed, while writing this paper, that the allocator propagation traits for `std::allocator<void>` do not match those for `std::allocator<T>`. It was decided not to provide a drive-by fix, as `std::allocator<void>` is not a true allocator, and so should not end up in code relying on those traits. If the decision is made not to deprecate this specialization though, it could be worth revisiting the issue, just to address consistency and a lack of surprise.

Additional Candidates

Several additional aspects of the library were looked at, but decided against a recommendation for deprecation at this point. They are listed here, with rationale, in case others are more motivated. With appropriate additions to the standard, these may become stronger deprecation candidates for the next standard.

If members of the Library Working Groups would like to see any of these additional libraries deprecated, the author is happy to

extend the proposal, but feels that they go beyond the remit of this paper deprecating only features where a preferable alternative is already available in the standard.

Reconsider `vector<bool>` Partial Specialization

There has been a long history of the `bool` partial specialization of `std::vector` not satisfying the container requirements, and in particular, its iterators not satisfying the requirements of a random access iterator. A previous attempt to deprecate this container was rejected for C++11, [N2204](#).

One of the reasons for rejection is that it is not clear what it would mean to deprecate a particular specialization of a template. That could be addressed with careful wording. The larger issue is that the (packed) specialization of `vector<bool>` offers an important optimization that clients of the standard library genuinely seek, but would no longer be available. It is unlikely that we would be able to deprecate this part of the standard until a replacement facility is proposed and accepted, such as [N2050](#). Unfortunately, there are no such revised proposals currently being offered to the Library Evolution Working Group.

Reconsider `is_literal` Trait

The `is_literal` type trait offers negligible value to generic code, as what is really needed is the ability to know that a specific construction would produce constant initialization. The core term of a *literal type* having at least one `constexpr` constructor is too weak to be used meaningfully.

However, the (already implemented) trait does no harm, and allows compile-time introspection for which core-language type-categories a given template parameter might satisfy. Until the Core Working Group retire the notion of a literal type, the corresponding library trait should be preserved.

One way forward would be to write a paper proposing to remove the term from the core language while deprecating/removing the type trait, but the author of this paper is not (yet) sufficiently motivated to author such a proposal.

Reconsider the Temporary Buffer APIs

Library clause 20.7.11 **[temporary.buffer]** provides an API intended to efficiently create a *small* amount of additional storage for local, short-term use. The inspiration behind the API is to use some magic to create a tiny buffer on the stack.

This API would be considered an incomplete thought were it proposed today. As a functional API it lacks exception safety if the function allocating the buffer leaks, yet we offer no RAII-like wrappers to promote safe use.

It has been suggested that all current implementation of this API actually do not perform a more efficient allocation than the regular `new` operator, and, if that is genuinely the case, we should seriously consider deprecating this facility. Otherwise, we should probably complete the design with an appropriate guard/wrapper class, and encourage vendors to deliver on missed optimization opportunities.

It is possible the need for this API could be overtaken by delivering on a genuine C++ API or data structure for allocating off the stack, such as was proposed by `std::dynarray` in [N3662](#). However, the Evolution Working Group has not been keen on further work in this direction, and the Arrays TS appears to have stalled, if not abandoned entirely.

Reconsider raw storage iterators

Like the temporary buffer API, the `raw_storage_iterator` (20.7.10 **[storage.iterator]**) and uninitialized algorithms (20.7.12 **[specialized.algorithms]**) are incomplete thoughts, although they appeared complete at the time of adoption.

One obvious shortcoming of the iterator adapter is that it could easily query the type of element the adapted iterator is prepared to construct via `iterator_traits` and provide a default template argument for the second parameter, simplifying use. That no such proposal has come forward in almost 20 years suggests that this is not a widely used component. Similarly, a type-deducing factory function would be a natural extension, especially with the advent of `auto` in C++11, yet no such proposal has materialized.

A deeper concern arises from the completion of allocator support in C++11. There is now a common requirement that constructing an object in a region of allocated memory should invoke the `construct` method of the corresponding allocator. This is not possible with the current iterator, and undermines its use as an implementation detail in users writing generic containers. However, no-one seems motivated to invest the time in proposing an allocator-aware raw storage iterator, nor adding allocator-awareness to the uninitialized memory algorithms.

Without a clear alternative to replace them, there is no clear deprecation path other than simply looking to drop support for the whole idea in some future standard. That goes beyond the intent of this paper.

Proposed Wording

Amend existing library clauses as below:

20.7.2 Header <memory> synopsis [memory.syn]

```
namespace std {
    // ...

    // 20.7.9, the default allocator:
    template <class T> class allocator;
template <> class allocator<void>;
    template <class T, class U>
        bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
    template <class T, class U>
        bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

    // ...
}
```

20.7.9 The default allocator [default.allocator]

1. All specializations of the default allocator satisfy the allocator completeness requirements 17.6.3.5.1.

```
namespace std {
template <class T> class allocator;

    // specialize for void:
template <> class allocator<void> {
    public:
    typedef void* pointer;
    typedef const void* const_pointer;
// reference-to-void members are impossible.
    typedef void value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
};

    template <class T> class allocator {
    public:
    typedef size_t size_type;
    typedef ptrdiff_t difference_type;
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef T& reference;
    typedef const T& const_reference;
    typedef T value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
    typedef true_type propagate_on_container_move_assignment;
    typedef true_type is_always_equal;
    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template <class U> allocator(const allocator<U>&) noexcept;
    ~allocator();
    pointer address(reference x) const noexcept;
    const_pointer address(const_reference x) const noexcept;
    pointerT* allocate(
        size_type, allocator<void>::const_pointer const void* hint = 0);
    void deallocate(pointerT* p, size_type n);
    size_type max_size() const noexcept;
    template<class U, class... Args>
        void construct(U* p, Args&&... args);
    template <class U>
        void destroy(U* p);
    };
}
```

20.7.9.1 allocator members [allocator.members]

1. Except for the destructor, member functions of the default allocator shall not introduce data races (1.10) as a result of concurrent calls to those member functions from different threads. Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before the next allocation (if any) in this order.

```
pointer address(reference x) const noexcept;
```

2. **Returns:** The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.

```
const_pointer address(const_reference x) const noexcept;
```

3. **Returns:** The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.

```
pointer T* allocate(size_type n, allocator::const_pointer const void* hint = 0);
```

4. [*Note:* In a container member function, the address of an adjacent element is often a good choice to pass for the `hint` argument. - *end note*]

5. **Returns:** A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (3.11).

6. **Remark:** the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called. The use of `hint` is unspecified, but intended as an aid to locality if an implementation so desires.

7. **Throws:** `bad_alloc` if the storage cannot be obtained.

```
void deallocate(pointer T* p, size_type n);
```

8. **Requires:** `p` shall be a pointer value obtained from `allocate()`. `n` shall equal the value passed as the first argument to the invocation of `allocate` which returned `p`.

9. **Effects:** Deallocates the storage referenced by `p`.

10. **Remarks:** Uses `::operator delete(void*, std::size_t)` (18.6.1), but it is unspecified when this function is called.

```
size_type max_size() const noexcept;
```

11. **Returns:** The largest value `N` for which the call `allocate(N, 0)` might succeed.

```
template <class U, class... Args>
void construct(U* p, Args&&... args);
```

12. **Effects:** `::new((void *)p) U(std::forward<Args>(args)...)`

```
template <class U>
void destroy(U* p);
```

13. **Effects:** `p->~U()`

Delete two rows from Table 101 - Associative container requirements (in addition to container)

23.2.4 Associative containers [associative.reqmts]

Expression	Return type	Assertion/note/pre-/post-condition	Complexity
		... rows elided ...	
<code>key_compare</code>	Compare	defaults to <code>less<key_type></code>	compile time
<code>value_compare</code>	a binary predicate type	is the same as <code>key_compare</code> for <code>set</code> and <code>multiset</code> ; is an ordering relation on pairs induced by the first component (i.e., <code>Key</code>) for <code>map</code> and <code>multimap</code> .	compile time
		... rows elided ...	
<code>a.key_comp()</code>	<code>X::key_compare</code>	returns the comparison objects out of which <code>a</code> was constructed	constant
<code>a.value_comp()</code>	<code>X::value_compare</code>	returns an object of <code>value_compare</code> constructed out of the comparison object	constant
		... rows elided ...	

23.4.4 Class template `map` [map]

23.4.4.1 Class template `map` overview [map.overview]

```
namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class map {
    public:
        // types
        // details elided...

        class value_compare {
        friend class map;
        protected:
            Compare comp;
            value_compare(Compare c) : comp(c) {}
        };
    };
};
```

```

public:
    typedef bool result_type;
    typedef value_type first_argument_type;
    typedef value_type second_argument_type;
    bool operator()(const value_type& x, const value_type& y) const {
        return comp(x.first, y.first);
    }
};

// 23.4.4.2, construct/copy/destroy:
// details elided...

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// remaining details elided...
};
}

```

23.4.5 Class template multimap [multimap]

23.4.5.1 Class template multimap overview [multimap.overview]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class multimap {
    public:
        // types
        // details elided...

        class value_compare {
        friend class multimap;
        protected:
            Compare comp;
            value_compare(Compare c) : comp(c) {}
        public:
            typedef bool result_type;
            typedef value_type first_argument_type;
            typedef value_type second_argument_type;
            bool operator()(const value_type& x, const value_type& y) const {
                return comp(x.first, y.first);
            }
        };

        // construct/copy/destroy:
        // details elided...

        // observers:
        key_compare key_comp() const;
        value_compare value_comp() const;

        // remaining details elided...
    };
}

```

23.4.6 Class template set [set]

23.4.6.1 Class template set overview [set.overview]

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class set {
    public:
        // types
        typedef Key          key_type;
        typedef Key          value_type;
        typedef Compare      key_compare;
        typedef Compare      value_compare;
        // details elided...

        // 23.4.6.2, construct/copy/destroy:
        // details elided...
    };
}

```

```

    // observers:
    key_compare key_comp() const;
    value_compare value_comp() const;

    // remaining details elided...
};
}

```

23.4.7 Class template multiset [multiset]

23.4.7.1 Class template multiset overview [multiset.overview]

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class multiset {
    public:
        // types
        typedef Key          key_type;
        typedef Key          value_type;
        typedef Compare      key_compare;
        typedef Compare      value_compare;
        // details elided...

        // construct/copy/destroy:
        // details elided...

        // observers:
        key_compare key_comp() const;
        value_compare value_comp() const;

        // remaining details elided...
    };
}

```

24.3 Header <iterator> synopsis [iterator.synopsis]

```

namespace std {
    // 24.4, primitives:
    template<class Iterator> struct iterator_traits;
    template<class T> struct iterator_traits<T*>;

    template<class Category, class T, class Distance = ptrdiff_t,
            class Pointer = T*, class Reference = T&> struct iterator;

    // Rest of header elided...
}

```

24.4.2 Basic iterator [iterator.basie]

1. The `iterator` template may be used as a base class to ease the definition of required types for new iterators.

```

namespace std {
    template<class Category, class T, class Distance = ptrdiff_t,
            class Pointer = T*, class Reference = T&>
    struct iterator {
        typedef T          value_type;
        typedef Distance   difference_type;
        typedef Pointer     pointer;
        typedef Reference   reference;
        typedef Category    iterator_category;
    };
}

```

25 Algorithms library [algorithms]

25.1 General [algorithms.general]

Header <algorithm> synopsis

```

#include <initializer_list>

namespace std {
    // 25.2, non-modifying sequence operations:

```

```

// ... several declarations omitted for brevity

template<class InputIterator1, class InputIterator2>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2);

template
<class InputIterator1, class InputIterator2, class BinaryPredicate>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2);

template
<class InputIterator1, class InputIterator2, class BinaryPredicate>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2,
         BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2);

template
<class InputIterator1, class InputIterator2, class BinaryPredicate>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2, InputIterator2 last2);

template
<class InputIterator1, class InputIterator2, class BinaryPredicate>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2, InputIterator2 last2,
           BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2, BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2, ForwardIterator2 last2,
                   BinaryPredicate pred);

// ... many more declarations omitted for brevity
}

```

25.2.10 Mismatch [mismatch]

```

template<class InputIterator1, class InputIterator2>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>

```

```

pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2);

template <class InputIterator1, class InputIterator2,
         class BinaryPredicate>
pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2,
         BinaryPredicate pred);

```

1. **Remarks:** If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.
2. **Returns:** A pair of iterators `i` and `j` such that `j == first2 + (i - first1)` and `i` is the first iterator in the range `[first1, last1)` for which the following corresponding conditions hold:
 1. `j` is in the range `[first2, last2)`.
 2. `!(i == *(first2 + (i - first1)))`
 3. `pred(*i, *(first2 + (i - first1))) == false`
 Returns the pair `first1 + min(last1 - first1, last2 - first2)` and `first2 + min(last1 - first1, last2 - first2)` if such an iterator `i` is not found.
3. **Complexity:** At most `min(last1 - first1, last2 - first2)` applications of the corresponding predicate.

25.2.11 Equal [alg.equal]

```

template<class InputIterator1, class InputIterator2>
bool equal(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
bool equal(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
bool equal(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
bool equal(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, InputIterator2 last2,
          BinaryPredicate pred);

```

1. **Remarks:** If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.
2. **Returns:** If `last1 - first1 != last2 - first2`, return false. Otherwise return true if for every iterator `i` in the range `[first1, last1)` the following corresponding conditions hold: `*i == *(first2 + (i - first1))`, `pred(*i, *(first2 + (i - first1))) != false`. Otherwise, returns false.
3. **Complexity:** No applications of the corresponding predicate if `InputIterator1` and `InputIterator2` meet the requirements of random access iterators and `last1 - first1 != last2 - first2`. Otherwise, at most `min(last1 - first1, last2 - first2)` applications of the corresponding predicate.

25.2.12 Is permutation [alg.is_permutation]

```

template<class ForwardIterator1, class ForwardIterator2>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  BinaryPredicate pred);

```

1. **Requires:** `ForwardIterator1` and `ForwardIterator2` shall have the same value type. The comparison function shall be an equivalence relation.

2. **Remarks:** If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.
3. **Returns:** If `last1 - first1 != last2 - first2`, return false. Otherwise return true if there exists a permutation of the elements in the range `[first2, first2 + (last1 - first1))`, beginning with `ForwardIterator2 begin`, such that `equal(first1, last1, begin)` returns true OR `equal(first1, last1, begin, pred)` returns true; otherwise, returns false.
4. **Complexity:** No applications of the corresponding predicate if `ForwardIterator1` and `ForwardIterator2` meet the requirements of random access iterators and `last1 - first1 != last2 - first2`. Otherwise, exactly `distance(first1, last1)` applications of the corresponding predicate if `equal(first1, last1, first2, last2)` would return true if `pred` was not given in the argument list or `equal(first1, last1, first2, last2, pred)` would return true if `pred` was given in the argument list; otherwise, at worst $O(N^2)$, where N has the value `distance(first1, last1)`.

Insert new clauses into Annex D:

D.w The default allocator [depr.default.allocator]

1. The following members are defined in addition to those specified in Clause 20:

```
namespace std {
    template <> class allocator<void> {
    public:
        typedef void* pointer;
        typedef const void* const_pointer;
        // reference-to-void members are impossible.
        typedef void value_type;
        template <class U> struct rebind { typedef allocator<U> other; };
    };

    template <class T> class allocator {
    public:
        typedef size_t size_type;
        typedef ptrdiff_t difference_type;
        typedef T* pointer;
        typedef const T* const_pointer;
        typedef T& reference;
        typedef const T& const_reference;
        template <class U> struct rebind { typedef allocator<U> other; };

        pointer address(reference x) const noexcept;
        const_pointer address(const_reference x) const noexcept;
        template<class U, class... Args>
            void construct(U* p, Args&&... args);
        template <class U>
            void destroy(U* p);
    };
}
```

```
pointer address(reference x) const noexcept;
```

2. Returns: The actual address of the object referenced by `x`, even in the presence of an overloaded operator`&`.

```
const_pointer address(const_reference x) const noexcept;
```

3. Returns: The actual address of the object referenced by `x`, even in the presence of an overloaded operator`&`.

```
template <class U, class... Args>
    void construct(U* p, Args&&... args);
```

4. Effects: `::new((void *)p) U(std::forward<Args>(args)...)`

```
template <class U>
    void destroy(U* p);
```

5. Effects: `p->~U()`

D.x Value Comparators for Associative Containers [depr.container.assoc.value compare]

1. The following member names are defined in addition to names specified in Clause 23. It is unspecified whether the `value_compare` classes are provided directly as member classes, or as aliases of classes with the same specification:

```
namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
```

```

class map {
public:
    class value_compare {
    friend class map;
    protected:
        Compare comp;
        value_compare(Compare c) : comp(c) {}
    public:
        typedef bool result_type;
        typedef value_type first_argument_type;
        typedef value_type second_argument_type;
        bool operator()(const value_type& x, const value_type& y) const {
            return comp(x.first, y.first);
        }
    };

    // observers:
    value_compare value_comp() const;
};
}

```

```

value_compare value_comp() const;

```

2. Returns: an object of type value_compare constructed out of the comparison object.

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class multimap {
    public:
        class value_compare {
        friend class multimap;
        protected:
            Compare comp;
            value_compare(Compare c) : comp(c) {}
        public:
            typedef bool result_type;
            typedef value_type first_argument_type;
            typedef value_type second_argument_type;
            bool operator()(const value_type& x, const value_type& y) const {
                return comp(x.first, y.first);
            }
        };

        // observers:
        value_compare value_comp() const;
    };
}

```

```

value_compare value_comp() const;

```

3. Returns: an object of type value_compare constructed out of the comparison object.

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class set {
    public:
        // types
        typedef Compare value_compare;

        // observers:
        value_compare value_comp() const;
    };
}

```

```

value_compare value_comp() const;

```

4. Returns: an object of type value_compare constructed out of the comparison object.

```

namespace std {
    template <class Key, class Compare = less<Key>,
              class Allocator = allocator<Key> >
    class multiset {
    public:
        typedef Compare value_compare;

        // observers:
        value_compare value_comp() const;
    };
}

```

```
};  
}
```

```
value_compare value_comp() const;
```

5. Returns: an object of type `value_compare` constructed out of the comparison object.

D.y Basic iterator [depr.iterator.basic]

1. The header `<iterator>` has the following addition:

```
namespace std {  
    template<class Category, class T, class Distance = ptrdiff_t,  
            class Pointer = T*, class Reference = T&>  
    struct iterator {  
        typedef T value_type;  
        typedef Distance difference_type;  
        typedef Pointer pointer;  
        typedef Reference reference;  
        typedef Category iterator_category;  
    };  
}
```

2. The iterator template may be used as a base class to provide the type alias members required for the definition of an iterator type.
3. [Note: If the new iterator type is a class template, then these aliases will not be visible from within the iterator class's template definition, but only to callers of that class - end note]

D.z Algorithms with Partially Formed Ranges [depr.algorithm.partial.range]

1. The following function template overloads are provided by the `<algorithm>` header in addition to those specified in Clause 25:

```
namespace std {  
    // non-modifying sequence operations:  
  
    template <class InputIterator1, class InputIterator2>  
    pair <InputIterator1, InputIterator2>  
    mismatch(InputIterator1 first1, InputIterator1 last1,  
            InputIterator2 first2);  
  
    template <class InputIterator1, class InputIterator2, class BinaryPredicate>  
    pair <InputIterator1, InputIterator2>  
    mismatch(InputIterator1 first1, InputIterator1 last1,  
            InputIterator2 first2, BinaryPredicate pred);  
  
    template <class InputIterator1, class InputIterator2>  
    bool equal(InputIterator1 first1, InputIterator1 last1,  
            InputIterator2 first2);  
  
    template <class InputIterator1, class InputIterator2, class BinaryPredicate>  
    bool equal(InputIterator1 first1, InputIterator1 last1,  
            InputIterator2 first2, BinaryPredicate pred);  
  
    template <class ForwardIterator1, class ForwardIterator2>  
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,  
            ForwardIterator2 first2);  
  
    template <class ForwardIterator1, class ForwardIterator2,  
            class BinaryPredicate>  
    bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,  
            ForwardIterator2 first2, BinaryPredicate pred);  
}
```

2. For each of the following algorithms, `last2` denotes an invented iterator object equivalent to `first2 + (last1 - first1)`. [Note: the behavior may undefined if there is no such reachable iterator — end note]

```
template <class InputIterator1, class InputIterator2>  
pair <InputIterator1, InputIterator2>  
mismatch(InputIterator1 first1, InputIterator1 last1,  
        InputIterator2 first2);
```

3. Effects: equivalent to `mismatch(first1, first2, last1, last2);`

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>  
pair <InputIterator1, InputIterator2>  
mismatch(InputIterator1 first1, InputIterator1 last1,
```

```
InputIterator2 first2, BinaryPredicate pred);
```

4. *Effects: equivalent to* `mismatch(first1, first2, last1, last2, pred);`

```
template <class InputIterator1, class InputIterator2>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2);
```

5. *Effects: equivalent to* `equal(first1, first2, last1, last2);`

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>
bool equal(InputIterator1 first1, InputIterator1 last1,
           InputIterator2 first2, BinaryPredicate pred);
```

6. *Effects: equivalent to* `equal(first1, first2, last1, last2, pred);`

```
template <class ForwardIterator1, class ForwardIterator2>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2);
```

7. *Effects: equivalent to* `is_permutation(first1, first2, last1, last2);`

```
template <class ForwardIterator1, class ForwardIterator2,
          class BinaryPredicate>
bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                   ForwardIterator2 first2, BinaryPredicate pred);
```

8. *Effects: equivalent to* `is_permutation(first1, first2, last1, last2, pred);`

Acknowledements

References

- [N3671](#) Making non-modifying sequence operations more robust: Revision 2, Mike Spertus, Attila Pall
- [P0090R0](#) Removing result_type, etc., Stephan T. Lavavej